

ECLAIR: Anytime-Valid Certification of LLM-Generated Constraint Models with Budget-Aware Probe Routing

Quang-Vinh Dang^{1*}

^{1*}British University Vietnam, Hung Yen, Vietnam.

Corresponding author(s). E-mail(s): vinh.dq4@buv.edu.vn;

Abstract

Large language models translate natural-language problem descriptions into formal constraint models, but they lack the mathematical guarantees required for formal correctness, and no ground truth exists against which a generated model can be checked — the oracle problem. Existing verification approaches are heuristic detectors: they report empirical detection rates but provide no statistical guarantee on the model one ultimately accepts, no principled stopping rule, and no way to decide which test to run next under a compute budget. We introduce ECLAIR, a sequential, anytime-valid, budget-aware certification layer. Each candidate model carries an e-process — a nonnegative supermartingale wealth process — betting against the hypothesis that the candidate is faithful, fed by three tiers of imperfect probes: exhaustively enumerated micro-instances, provably valid metamorphic relations, and cross-model majority checks. Conservative calibration of the probes’ null alarm rates preserves validity under the composite null, and a Kelly-style rule routes solver time to the candidate–probe pair with the highest expected evidence growth per second. Ville’s inequality bounds the probability of ever falsely rejecting a faithful model by α under optional stopping, and e-BH selection controls the false discovery rate over the whole generate–test–repair tournament. On a mutation testbed of five problem families with real CP-SAT probes, the empirical false-rejection rate is zero at every nominal level while **87–94%** of injected faults are detected; an end-to-end experiment with two LLM families reproduces the guarantee on real generated models.

Keywords: constraint modelling, large language models, e-processes, anytime-valid inference, solver-in-the-loop verification, false discovery rate

1 Introduction

Constraint programming’s long-standing adoption bottleneck is the expertise barrier: translating a problem description held in natural language into a formal model. Large language models (LLMs) now cross that barrier routinely — generating CPMpy, MiniZinc, or MILP models from plain-English specifications [1–3] — but they do so without any intrinsic guarantee of correctness. A generated model that compiles and solves is not necessarily the model that was asked for: a misread coefficient, a dropped constraint family, or a flipped inequality produces a plausible artifact that returns confidently wrong optima. Because the specification exists only in natural language, there is no oracle to compare against; this is the central verification difficulty the field currently faces.

The question a practitioner asks of any verification layer is: *what guarantee do I get on the model I finally accept?* Existing answers are heuristic. Detection frameworks based on metamorphic relations or mutation-style testing report empirical detection rates, but offer no control over the probability of certifying an unfaithful model, no principled rule for when enough testing has been done, and no guidance on which of many possible tests — differing wildly in cost and power — to run next under a finite solver budget. Fixed-sample statistical tools do not repair this: a generate–test–repair loop inherently peeks at accumulating evidence and adapts, which invalidates fixed-sample p-values and split-conformal calibration alike.

A concrete miniature fixes ideas. A dispatcher asks for “a knapsack model: pick items within the truck’s 120 kg limit, maximizing value; items 3 and 7 cannot travel together.” A language model returns clean CPMpy code — but read 120 as the *count* bound of a different template, wired the weight vector where values belong, or emitted `sum(w*x) >= cap` after seeing a covering example in its context. Each of these artifacts compiles, solves in milliseconds, and returns a confident number. Asking a second model and comparing helps only if their errors differ; testing on a hand-checkable toy instance helps only if the error manifests there; and the practitioner who keeps testing “until it looks fine” has silently destroyed the error control of whatever test they were using. What is missing is an accounting system for the accumulated evidence — one that survives the practitioner’s own adaptivity.

This paper introduces **ECLAIR** (E-process Certification of LLM-generated constraint models with Adaptive Instance Routing), a certification layer built on the game-theoretic statistics of e-processes [4–6]. The design follows three observations.

First, *optional stopping is the workflow*. E-processes are designed for continuous monitoring: rejecting whenever the wealth process crosses $1/\alpha$ controls the type-I error at α uniformly over all stopping times (Ville’s inequality [7]), so the loop may stop the moment evidence suffices — a direct compute saving — and may resume testing at any time without penalty.

Second, *probes are heterogeneous, imperfect, and dependent*. A brute-forced micro-instance is a near-perfect oracle but expensive; a metamorphic relation is provably false-alarm-free but weak; a cross-model agreement check is cheap, noisy, and entangled with the candidate pool. E-values multiply across adaptively chosen, dependent probes as long as each factor is a valid e-variable conditionally on the past, and we show that a conservative upper confidence bound on each probe family’s null alarm rate —

rather than an unattainable exact null distribution — suffices for validity under the composite null “this candidate is faithful.”

Third, *the budget question is a portfolio question*. Choosing which (candidate, probe) pair to run next to maximize expected log-evidence growth per solver-second is Kelly-optimal evidence acquisition [6, 8]; because the routing decision is measurable with respect to the past, it affects power only, never validity.

Contributions.

(i) A formal certification protocol for LLM-generated constraint models: per-candidate e-processes over a three-tier probe pool, with hard certificates (probes whose false-alarm probability is exactly zero by proof) embedded as infinite e-values (Section 4). (ii) Validity theory for the composite null: conservative calibration preserves the supermartingale property (Lemma 1), yielding anytime-valid false-rejection control under arbitrary predictable routing (Theorem 2), false discovery rate control over the full generate–test–repair tournament via e-BH [9] (Proposition 3), and a budget bound on the expected solver time to reject an unfaithful candidate (Proposition 4). (iii) A verified probe library for five problem families — knapsack with conflicts, set cover, graph colouring, generalized assignment, and on-time job selection — including per-family metamorphic relations proved valid at specification level (Proposition 6). (iv) An open implementation on CPMpy [10] and OR-Tools CP-SAT [11] and a five-part empirical study (Section 7): a mutation testbed in which validity holds exactly at every nominal level while 87–94% of injected faults are caught; ablations quantifying the contribution of each probe tier, including an empirical demonstration that pool-entangled probes consume the conservative validity margin; a matched-budget comparison in which the practitioner’s alternatives either cannot reject at all (single-shot, self-consistency voting) or break their nominal level (a fixed-sample test at 0.061, its uncorrected peeking variant at 0.075) while ECLAIR detects 0.905 at exactly zero false rejections; an end-to-end experiment with two LLM families in which every crash-free generated model is certified without a single false rejection; and an ambiguity study whose honest negative result — current models normalize boundary-language ambiguity to the conventional reading — sharpens where the silent-error regime actually lives.

2 Related work

LLMs for constraint modelling.

In-context learning pipelines translate natural-language descriptions into CPMpy models with retrieval and self-refinement [1]; benchmark suites evaluate such systems on CSPLib-derived corpora [12]. Agentic systems iterate modelling with solver feedback: CP-Agent [3] maintains a persistent Python session around CPMpy, and MCP-Solver [13] exposes constraint solvers to LLMs through a model context protocol. ConstraintLLM [2] targets industrial-level CP with a neuro-symbolic architecture. All these systems *generate*; ECLAIR is complementary — it decides, with a guarantee, whether to *trust* what was generated, and can wrap any of them.

Testing lineage.

Two software-testing traditions supply ECLAIR’s probe vocabulary. Metamorphic testing [14] addresses the oracle problem by checking necessary relations between outputs on related inputs — exactly our tier B, with the strengthening that relations proved against the *specification* of an optimization problem have null alarm probability zero, turning a testing heuristic into a hard certificate inside a statistical process. Mutation testing [15, 16] contributes the evaluation methodology: systematically injected faults with screened non-equivalence are the standard way to measure a test suite’s power when real faults are scarce, and our functional-corruption testbed (Section 7) is its adaptation to model faithfulness — with the twist that here the “test suite” carries a statistical guarantee, so the mutation score is a *power* estimate under a controlled *level*, not a standalone adequacy number.

Verifying generated models.

Two recent lines attack verification directly: falsification frameworks deriving provably valid metamorphic relations from duality and monotonicity theory, in which a violated relation certifies unfaithfulness, and agent-based frameworks that compose test cases and apply optimization-oriented mutation testing.¹ ECLAIR subsumes the binary verdicts of such detectors as the special case of an infinite e-value, and adds what they lack: a guarantee on the accepted model, a stopping rule, FDR control across repairs, and budget-aware test selection.

Anytime-valid inference.

E-values and e-processes provide measures of evidence valid at arbitrary stopping times [4, 5, 17]; safe tests and the GROW criterion connect optimal evidence growth to log-optimal (Kelly) betting [6, 8]; betting-based confidence sequences supply the adaptive wagering machinery [18]; and the e-BH procedure controls FDR under arbitrary dependence [9]. ECLAIR ports this toolkit into constraint-model verification, where, to our knowledge, sequential anytime-valid certification has not been attempted.

Why not fixed-sample or conformal tools.

It is worth being precise about why the natural statistical alternatives do not fit this workflow, since the certification problem looks superficially like ordinary hypothesis testing or distribution-free calibration. A fixed-sample test requires the number and identity of probes to be fixed before any outcome is seen; a generate–test–repair loop violates this at every step — it looks at the running evidence to decide whether to continue testing, which probe to run next, whether to repair, and when to stop, and each of those choices invalidates a fixed-sample p-value in the familiar optional-stopping way. Split-conformal calibration instead requires exchangeability between the calibration data and the test stream; here the “test stream” is adaptively generated (repaired candidates are chosen *because* of earlier outcomes; probe instances are routed by an adaptive policy), so exchangeability fails by construction. E-processes are designed for exactly this regime: validity is preserved under any data-dependent stopping, continuation, and selection rule, because the guarantee is a supermartingale property rather

¹Full citations pending verification.

than a sampling-distribution property [4]. The practical consequence is the one that matters operationally: the certifier may spend its budget *where the evidence points*, stop the moment a decision is supported, and still quote the same α .

3 The certification problem

A specification s is a natural-language description of a parameterized combinatorial optimization problem, together with an instance distribution $\mathcal{D}$ over parameter values. A *candidate* m is an executable function mapping an instance $I \sim \mathcal{D}$ to a formal model $m(I)$ whose exact optimum we write $v_m(I) \in \mathbb{Z} \cup \{\text{INFEAS}\}$. The (unknown) faithful value $v^*(I)$ is the optimum of the problem s actually describes. Candidate m is *faithful* if $v_m(I) = v^*(I)$ for $\mathcal{D}$ -almost-every I .

Given candidates $m_1, \dots, m_k$ produced by an LLM ensemble, a compute budget B (solver-seconds), and a risk level α , the certifier must decide which candidates to reject, which (if any) to certify, or whether to abstain — without access to v^* . The null hypothesis attached to each candidate is

$$H_0(m) : \quad m \text{ is faithful.}$$

$H_0(m)$ is *composite*: faithfulness does not determine the law of any probe outcome, which also depends on the instance generator, on checker noise, and (for cross-model probes) on the composition of the surviving pool. All guarantees below therefore hold uniformly over the null class, which we make precise through conditional bounds on probe alarm rates.

4 The ECLAIR framework

Figure 1 shows the architecture. An LLM ensemble proposes candidates; a cheap intake gate discards code that cannot build and solve a toy instance; every surviving candidate carries its own e-process; a router spends the solver budget one probe at a time; and three exits are possible — rejection (with a counterexample that can seed a repaired candidate), certification of a survivor by e-BH selection, or abstention with the full evidence ledger. The subsections follow the data flow.

4.1 Probe tiers

A *probe* is a randomized procedure that consumes solver time and returns a binary alarm $X \in \{0, 1\}$ against a designated candidate. ECLAIR uses three tiers.

Tier A: micro-instance oracles.

Draw an instance small enough for exhaustive enumeration of the specification’s decision space using an executable feasibility checker, and alarm iff the candidate’s exact optimum differs from the enumerated optimum. With an exact checker the null alarm probability is zero; in general the checker itself is imperfect (in a fully LLM-generated pipeline it is LLM-compiled and validated by cross-agreement), so tier A is treated as a calibrated noisy probe.

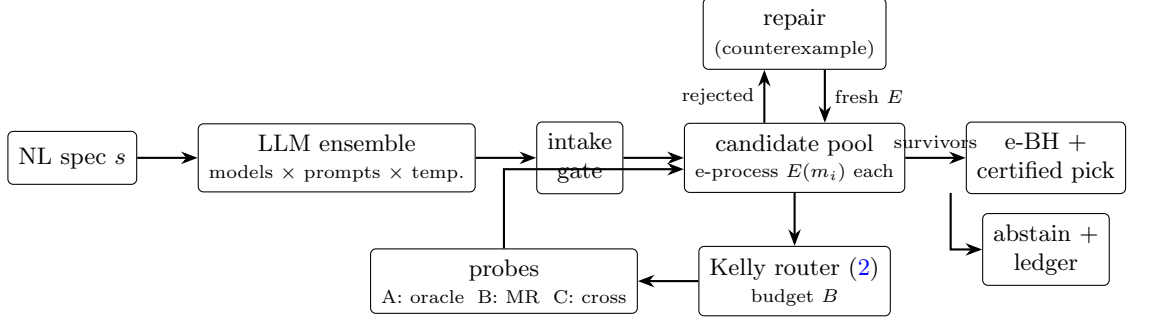


Fig. 1 ECLAIR’s architecture. Every candidate carries its own wealth process; the router spends the shared budget one probe at a time; rejection feeds repair; terminal e-values enter e-BH; if nothing survives, the evidence ledger explains why

Tier B: metamorphic relations (hard certificates).

A metamorphic relation (MR) transforms an instance I into I' such that the specification provably implies a relation between $v^*(I)$ and $v^*(I')$ — e.g. enlarging a knapsack capacity cannot decrease the optimum, deleting a graph edge cannot increase the chromatic number, consistently permuting item data leaves the optimum unchanged. The probe solves the *candidate* on both sides exactly and alarms iff the relation is violated. For a faithful candidate the alarm probability is exactly zero: both solves return true optima of instances of the true specification, and the relation holds for the specification by proof. A tier-B alarm is therefore a *hard certificate* of unfaithfulness.

Tier C: cross-model majority checks.

Solve the designated candidate and up to three surviving partners on a fresh mid-size instance (beyond enumeration reach — this tier’s purpose); alarm iff at least two partners agree on a value and the candidate deviates from that majority. Tier C is cheap and noisy, and its null alarm rate depends on the pool: a faithful candidate alarms when a wrong majority forms. This entanglement is handled by calibration (Section 4.2) and is quantified empirically in Section 7.3.

4.2 Betting and conservative calibration

Let $\mathcal{F}_{t-1}$ denote everything observed before the t -th probe. For a tier-B probe the e-factor is $e_t = 1$ on pass and $e_t = \infty$ on alarm. For tiers A and C, ECLAIR bets a Bernoulli likelihood ratio

$$e_t = \left(\frac{p_1}{\bar{p}_0} \right)^{X_t} \left(\frac{1-p_1}{1-\bar{p}_0} \right)^{1-X_t}, \quad (1)$$

where $\bar{p}_0$ is a *conservative* (upper confidence) bound on the tier’s null alarm rate and $p_1 \in (\bar{p}_0, 1)$ a working detection rate; both come from a held-out calibration corpus of known-faithful models and screened mutants, with $\bar{p}_0$ the Clopper–Pearson upper bound [19] at confidence 97.5%. Only $\bar{p}_0$ matters for validity; p_1 tunes power.

Lemma 1 (Conservative bets are safe) *Let $X \in \{0, 1\}$ satisfy $\Pr(X = 1 \mid \mathcal{F}_{t-1}) \leq \bar{p}_0$ almost surely under H_0 , and let e be given by (1) with $0 < \bar{p}_0 < p_1 < 1$. Then $\mathbb{E}[e \mid \mathcal{F}_{t-1}] \leq 1$ under H_0 .*

Proof With $q = \Pr(X=1 \mid \mathcal{F}_{t-1})$, $\mathbb{E}[e \mid \mathcal{F}_{t-1}] = q \frac{p_1}{\bar{p}_0} + (1 - q) \frac{1-p_1}{1-\bar{p}_0}$, which is affine and increasing in q because $p_1/\bar{p}_0 > 1 > (1-p_1)/(1-\bar{p}_0)$. At $q = \bar{p}_0$ it equals $p_1 + (1-p_1) = 1$; hence for $q \leq \bar{p}_0$ it is at most 1. $\square$

The candidate’s wealth process is the running product $E_t(m) = \prod_{s \leq t, s \rightarrow m} e_s$ over the probes routed to m , with $E_0(m) = 1$.

The betting arithmetic, numerically.

With the calibrated values of Section 7 (Table 3), the multipliers of (1) are concrete: a tier-A alarm multiplies wealth by $p_1/\bar{p}_0 = 0.473/0.0153 \approx 31$, a pass by $(1-p_1)/(1-\bar{p}_0) \approx 0.53$; a tier-C alarm multiplies by ≈ 3.0 , a pass by ≈ 0.61 ; tier B multiplies by 1 on pass and rejects outright on alarm. Against the rejection threshold $1/\alpha = 20$ this means: a single tier-A alarm brings a fresh candidate to within one small step of rejection ($\log 31 > \log 20$ already, from wealth 1); two early tier-A passes push a candidate’s wealth down to ≈ 0.28 , after which the Kelly numerator of (2) for noisy tiers turns negative and the router parks the candidate on zero-bleed tier-B probes — exactly the flattening visible in Figure 2. The asymmetry (alarm evidence $\sim 31\times$, pass evidence $\sim 0.53\times$) is not a tuning choice but the likelihood-ratio geometry of a rare-under-null, common-under-alternative event.

4.3 Budget-aware routing

At each step ECLAIR selects

$$(m, p)^\star = \arg \max_{(m, p)} \frac{\mathbb{E}[\log e \mid q_m]}{c(p)}, \quad (2)$$

the candidate–probe pair maximizing expected log-wealth growth per solver-second: Kelly-optimal evidence acquisition in the sense of the GROW criterion [6, 8]. Here q_m is a running (Bayes) posterior that m is unfaithful, maintained with the same calibrated rates and used *only* for routing; $c(p)$ is the tier’s calibrated mean cost. For tiers A/C the numerator is $q_m G_1 + (1 - q_m) G_0$ with G_1, G_0 the expected log-factors of (1) under alarm rates $p_1, \bar{p}_0$; for tier B it is $q_m \delta_B (\log(1/\alpha) - \log E_t(m))$, the calibrated hard-certificate rate times the remaining log-gap to the rejection threshold. Because (2) is $\mathcal{F}_{t-1}$ -measurable, routing affects power only — validity is untouched (Theorem 2).

4.4 Decisions, repair, and abstention

Candidate m is *rejected* the first time $E_t(m) \geq 1/\alpha$ (or on a tier-B alarm). A rejected candidate may be *repaired*: the violating probe is fed back to the LLM as a counterexample and the repaired candidate enters the pool as a fresh e-process. When the budget is exhausted (or one candidate’s survival dominates), the terminal e-values of

Algorithm 1 ECLAIR

Require: spec s , candidates $m_1:k$, budget B , level α , calibrated $(\bar{p}_0, p_1, \delta_B, c)$ per tier

- 1: $E(m_i) \leftarrow 1$, $q_{m_i} \leftarrow q_{\text{prior}}$ for all i
- 2: **while** $B > 0$ **and** some candidate alive **do**
- 3: $(m, p) \leftarrow$ maximizer of (2) over alive candidates and tiers
- 4: run probe p on m ; observe alarm X , measured cost c ; $B \leftarrow B - c$
- 5: **if** p is tier B **and** $X = 1$ **then** reject m $\triangleright$ hard certificate
- 6: **else**
- 7: $E(m) \leftarrow E(m) \cdot e(X)$ by (1); update q_m by Bayes
- 8: **if** $E(m) \geq 1/\alpha$ **then** reject m
- 9: **end if**
- 10: **end if**
- 11: optionally: repair rejected m via counterexample; add as fresh e-process
- 12: **end while**
- 13: **return** e-BH rejection set at level α ; certified pick $\arg \min_{\text{survivors}} E(m)$, or abstain with the evidence ledger

all candidates ever considered enter the e-BH procedure [9] at level α ; among Ville-survivors the candidate with the smallest e-value is the certified output, together with its evidence ledger. If no candidate survives, ECLAIR *abstains* and returns the ledger — a structured explanation of which probes killed every reading of the specification, which doubles as a signal for conversational re-elicitation. Algorithm 1 summarizes the loop.

4.5 A worked run

Figure 2 traces one certified pool — three faithful candidates and three screened mutants of the set-cover family — under Kelly routing at $\alpha = 0.05$ with a 640 ms budget of measured solver time. Every mechanism of the framework is visible in the paths. Two mutants are driven across the rejection threshold within 60 ms: the router’s prior makes every candidate equally suspect at the start, tier-A alarms multiply their wealth by $p_1/\bar{p}_0 \approx 31$, and two to three alarms suffice. The third mutant survives its first probes by luck, its posterior q_m drops, and the router deprioritizes it — until the pool thins, its relative score rises again, and a single tier-B hard certificate (the near-vertical jump at ~ 290 ms) removes it. The faithful candidates’ paths tell the validity story: early tier-A/C passes drive their wealth *down* (a pass multiplies by $(1 - p_1)/(1 - \bar{p}_0) < 1$), after which the router — seeing low q_m — feeds them almost exclusively zero-bleed tier-B probes, so their wealth flattens far below the threshold and the remaining budget is spent where Lemma 1 makes passes free. The certified pick is the faithful candidate with the smallest terminal wealth; its ledger records every probe shown in the figure. Table 1 lists the first fourteen ledger entries verbatim: the entire decision logic of the framework — likelihood ratios, threshold crossings, router hand-offs to the zero-bleed tier — is legible in fourteen rows of arithmetic.

Table 1 The first fourteen ledger entries of the run in Figure 2 (m_1 – m_3 faithful, m_4 – m_6 mutants; $\alpha = 0.05$, rejection at $\log E \geq \log 20 \approx 3.00$). Steps 4–5: one tier-A alarm each rejects m_4, m_5 outright ($\log 31.0 = 3.43$). The lucky mutant m_6 passes its first probes and is parked, with the faithful, on zero-bleed tier-B probes (steps 11–14) — it is rejected much later by a hard certificate (Fig. 2).

Step	Candidate	Tier	e -factor	$\log E$ after
1–3	m_1, m_2, m_3	A (pass)	0.535	−0.626
4	m_4	A (alarm)	31.0	3.435 → reject
5	m_5	A (alarm)	31.0	3.435 → reject
6	m_6	A (pass)	0.535	−0.626
7–10	m_1, m_2, m_3, m_6	A (pass)	0.535	−1.252
11–14	m_1, m_2, m_3, m_6	B (pass)	1.000	−1.252

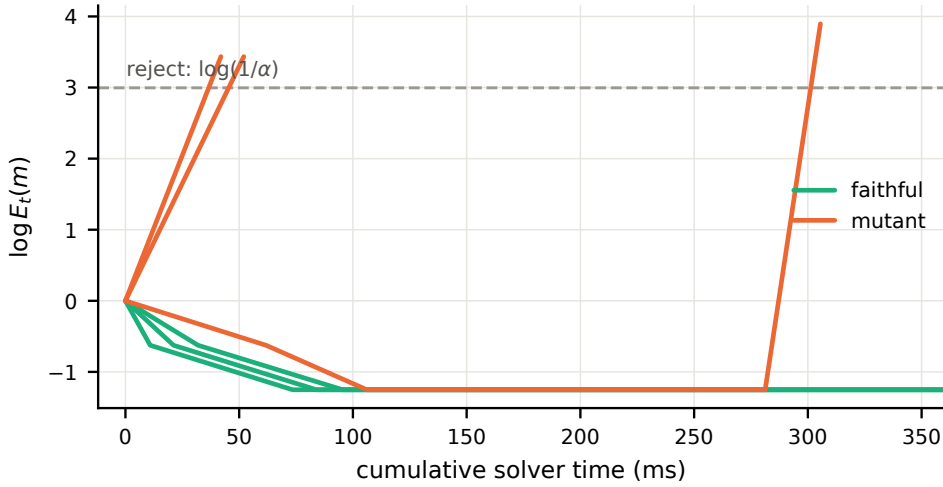


Fig. 2 Wealth trajectories $\log E_t(m)$ of one certified pool (three faithful set-cover candidates, three screened mutants; Kelly routing, $\alpha = 0.05$). Mutant paths cross the rejection threshold (dashed); the late near-vertical jump is a tier-B hard certificate. Faithful paths drift down on early noisy passes, then flatten as the router shifts them to zero-bleed tier-B probes

5 Guarantees

Assumption 1 Under $H_0(m)$: tier-B probes alarm with probability zero, and each tier-A/C probe routed to m alarms with conditional probability at most its calibrated bound $\bar{p}_0$, given $\mathcal{F}_{t-1}$.

The tier-B half of Assumption 1 is a theorem for the probe library of Section 7 (Proposition 6); the tier-A/C half is what conservative calibration targets, and its empirical status — including where it becomes delicate — is examined in Section 7.3.

Theorem 2 (Anytime validity) *Under Assumption 1, for any predictable routing rule, any repair policy, and any stopping rule,*

$$\Pr_{H_0(m)} (\exists t : E_t(m) \geq 1/\alpha) \leq \alpha.$$

Proof Each factor contributed to $E(m)$ is nonnegative and, by Lemma 1 (tiers A/C) or by the zero-alarm property (tier B, factor $\equiv 1$ a.s. under H_0), satisfies $\mathbb{E}[e_t | \mathcal{F}_{t-1}] \leq 1$; predictability of the routing rule makes “which probe, on which candidate” a $\mathcal{F}_{t-1}$ -measurable choice, so the tower property gives $\mathbb{E}[E_t(m)] \leq 1$ and $E_t(m)$ is a nonnegative supermartingale. Ville’s inequality [7] concludes. $\square$

Proposition 3 (Tournament FDR control) *Let $\{E(m)\}$ be the terminal e-values of all candidates considered across all generation and repair rounds. Applying e-BH at level α controls the false discovery rate of the rejected set at α , regardless of the dependence induced by shared probes, common instances, pool entanglement, or data-driven repairs.*

Proof Each $E(m)$ is a valid e-value for $H_0(m)$ by Theorem 2 (optional stopping preserves $\mathbb{E} \leq 1$). The e-BH procedure controls FDR for arbitrarily dependent e-values [9]. $\square$

Proposition 4 (Budget to rejection) *Fix an unfaithful candidate m and a routing rule under which the probes applied to m satisfy $\mathbb{E}[\log e_t | \mathcal{F}_{t-1}] \geq \Delta > 0$ and $|\log e_t| \leq L$. Then the number τ of probes until rejection satisfies $\mathbb{E}[\tau] \leq (\log(1/\alpha) + L)/\Delta$.*

Proof $M_t = \log E_t(m) - t\Delta$ is a submartingale minorant; optional stopping for the bounded-increment stopped process at $\tau \wedge n$ gives $\Delta \mathbb{E}[\tau \wedge n] \leq \mathbb{E}[\log E_{\tau \wedge n}] \leq \log(1/\alpha) + L$, and monotone convergence lets $n \rightarrow \infty$. $\square$

Proposition 4 is the quantitative content of “stop the moment evidence suffices”: the budget scales as $\log(1/\alpha)$, so tightening the guarantee is cheap — consistent with the flat detection curves across α observed in Section 7.2.

Finally, the tier-selection layer of the router admits a no-regret guarantee. The deployed router uses the plug-in posterior form (2); the following analyzes its online-learning relative, in which the tier weights are maintained by the exponentially weighted average (Hedge) forecaster [20, 21] over the realized growth-per-cost rewards.

Proposition 5 (No-regret tier selection) *Fix a candidate and let $r_t(p) = \log e_t(p)/c(p) \in [-L, L]$ denote the (bounded) realized log-growth per unit cost had tier $p \in \{A, B, C\}$ been probed at step t . Selecting tiers by Hedge with learning rate $\eta = \sqrt{8 \ln 3/T}$ guarantees*

$$\max_p \sum_{t=1}^T r_t(p) - \mathbb{E} \left[\sum_{t=1}^T r_t(p_t) \right] \leq L \sqrt{(T/2) \ln 3},$$

i.e. the realized evidence growth per unit cost is within $O(\sqrt{T})$ of the best fixed tier in hindsight; since tier selection remains $\mathcal{F}_{t-1}$ -measurable, Theorem 2 is unaffected.

Proof Immediate from the standard Hedge regret bound for bounded rewards over 3 experts [21, Thm. 2.2], after rescaling r_t to $[0, 1]$; predictability holds because the Hedge weights at step t depend only on rewards observed before t . $\square$

Proposition 6 (Probe-family validity) *For the five problem families of Section 7, the implemented metamorphic relations — capacity/deadline/cost monotonicity, structural relaxation (conflict, edge, or coverage-row deletion), restriction (edge addition), and consistent data permutation — have null alarm probability exactly zero when both sides are solved exactly.*

Proof sketch Each relation is proved at specification level; exact solves transfer it to any faithful candidate. Monotone relaxations follow from feasible-set inclusion (e.g. enlarging a knapsack capacity or deleting a conflict pair only enlarges the feasible set of the specification, so the maximum cannot decrease); objective monotonicity follows from pointwise dominance; permutation invariance follows from the specification’s symmetry under consistent relabeling of items, sets, vertices, or jobs. The one delicate case is deadline extension in on-time job selection, where the constraint *indices* shift with the deadline order: there, validity follows because the earliest-due-date feasibility conditions are equivalent to true single-machine schedulability of the selected set, and schedulability is monotone in deadlines. Full proofs accompany the probe library. $\square$

Remark 1 Assumption 1 for tier C deserves emphasis: the null alarm event “a wrong majority forms among partners” depends on the surviving pool, so its conditional probability is bounded uniformly only if calibration reflects the pool compositions actually encountered. Section 7.3 shows empirically that this is not pedantry — the conservative margin visibly erodes exactly, and only, when tier C is made the sole evidence source.

6 A verified probe library for five problem families

The guarantees of Section 5 are only as real as Assumption 1, and tier B’s half of that assumption is a per-relation *proof obligation*. This section discharges it for a concrete library spanning five combinatorial families chosen to exercise packing, covering, colouring, assignment, and scheduling structure. For each family we implement a reference model, an independent executable checker (the tier-A enumeration oracle), and the metamorphic relations below; full validity proofs are in Appendix A. Table 2 summarizes.

Three structural remarks.

First, the relations fall into three provable classes — *feasible-set monotonicity* (relaxing a resource or deleting a structural restriction enlarges the specification’s feasible set), *objective dominance* (a pointwise-increased cost vector cannot lower a minimum), and *symmetry* (consistent relabeling of indexed data is a bijection of the feasible set preserving objectives). These classes, not the individual relations, are what transfers to new families: any specification whose feasible set is downward-closed in a resource parameter inherits the first class verbatim, which is how the library should be extended

Table 2 The probe library. MRs state a provable relation between the specification optima of an instance I and its transform I' ; $\uparrow$ ($\downarrow$) means the optimum cannot decrease (increase), $=$ means it is unchanged. Mutation classes are the functional corruptions used to create ground-truth unfaithful candidates.

Family (sense)	Metamorphic relations	Mutation classes
Knapsack with conflicts (max)	capacity+ δ : $\uparrow$; drop conflict: $\uparrow$; permute items: $=$	capacity shift; weight/value entry; vector rotation; conflict-row drop/half-drop; operator flip
Set cover (min)	drop element: $\downarrow$; cost entry+ δ : $\uparrow$; permute sets: $=$	cost entry; coverage-row drop/half-drop; operator flip
Graph colouring (min)	drop edge: $\downarrow$; add edge: $\uparrow$; relabel vertices: $=$	edge drop/half-drop; adjacency-logic flip
Generalized assignment (min)	capacity+ δ : $\downarrow$; cost entry+ δ : $\uparrow$; permute tasks: $=$	size/capacity entry; vector rotation; cost-matrix entry; operator flip
On-time job selection (max)	deadline+ δ : $\uparrow$; weight+ δ : $\uparrow$; processing- δ : $\uparrow$; permute jobs: $=$	processing/deadline/weight entry; vector rotation; operator flip

toward global constraints such as **alldifferent** (invariant under value permutation) and **cumulative** (monotone in capacity and in deadline vectors).

Second, one relation is genuinely delicate and is the reason proofs are owed rather than waved at: deadline extension in on-time selection changes not only a right-hand side but the *index sets* of the earliest-due-date constraints, so monotonicity of the constraint system is not syntactically obvious; validity is restored by the classical equivalence between the EDD inequalities and true single-machine schedulability, which *is* monotone in deadlines (Appendix A). Metamorphic relations must be proved against the specification, not read off the formulation — this is precisely the failure mode that turns a would-be hard certificate into a silent source of false rejections.

Third, the mutation classes mirror the errors observed from real LLM modellers (Section 7.5) at the semantic rather than syntactic level: misread constants, positionally corrupted data, misaligned columns (rotation), dropped constraint structure, and inverted comparisons. Each corruption is *instance-generic* — applied identically to every instance the candidate ever sees — so a mutant is a wrong *model*, not a wrong instance, exactly like a systematic LLM misreading.

7 Experimental evaluation

Implementation.

All experiments run on CPMpy [10] with the OR-Tools CP-SAT solver [11]. Five problem families are implemented, each with a reference model builder, an *independent* pure-Python specification checker (the tier-A enumeration oracle), micro/mid instance generators, and the family’s metamorphic relations of Proposition 6: 0/1 knapsack

Table 3 Calibration corpus (two held-out families, 240 trials per tier and hypothesis). $\hat{p}_0/\hat{p}_1$ are observed alarm rates on faithful/mutant candidates; $\bar{p}_0$ is the Clopper–Pearson 97.5% upper bound the bets use; cost is measured mean solver time per probe.

Tier	$\hat{p}_0$ (faithful)	$\bar{p}_0$	$\hat{p}_1$ (mutants)	mean cost
A (micro-oracle)	0/240	0.0153	0.592	5.2 ms
B (MRs)	0/240	—	0.042	6.3 ms
C (cross-majority)	28/240	0.164	0.567	12.4 ms

with pairwise conflicts, minimum-cost set cover, graph colouring (minimizing the number of colours), generalized assignment, and weighted on-time job selection on a single machine. Probe costs are measured wall-clock solver-seconds. Code, calibration corpora, all raw results, and the cached LLM generations are provided as supplementary material.

Unfaithful candidates by functional mutation.

To measure error rates one needs ground truth, so the first two studies replace the LLM with *functional corruptions* of the reference builder — systematic, instance-generic errors of the kinds LLM modellers actually make: a bound misread by a constant, one vector entry corrupted positionally, a data-wiring misalignment (vector rotation), a dropped structural row or half-family, a corrupted cost-matrix entry, and a flipped comparison operator in one constraint family. Mutants are screened for semantic difference from the reference (equivalent mutants are resampled), the standard mutation-testing practice.

Calibration.

Betting parameters are calibrated on two held-out families (knapsack with conflicts, graph colouring) and evaluated on the three *disjoint* remaining families — so every result below also tests whether conservative calibration transfers across problem families. Table 3 reports the measured corpus. Two entries deserve comment. Tier B’s 0/240 row is not an estimate but a *consistency check* of Proposition 6 — a single faithful alarm would indicate an invalid relation and aborts calibration. Tier C’s null alarm rate of 11.7% is the empirical face of pool entanglement: even a faithful candidate is outvoted when two of its three sampled partners happen to agree on a wrong value; the Clopper–Pearson bound $\bar{p}_0 = 0.164$ is what the bets then use.

7.1 Mutation testbed: validity, power, and routing

Pools of $k = 6$ candidates (each faithful or a screened mutant with probability 1/2) are certified at $\alpha = 0.05$ under a shared budget of 640 ms of measured solver time per pool, 100 replications per routing policy with identical pools across policies. Table 4 reports the results.

Validity holds exactly — not one of ~ 900 faithful-candidate exposures across the three policies was rejected — despite the bets being calibrated on different problem

Table 4 Mutation testbed (100 replications/policy, $\alpha = 0.05$, real CP-SAT probes; calibration and evaluation families disjoint). False rejection is over faithful candidates ($n \approx 300$ per policy); detection over mutants; kill cost is mean solver time spent on a rejected mutant; pick accuracy is the probability the certified output is faithful.

Routing	False rej.	Detection	Kill cost	Pick acc.	e-BH FDR
Kelly (ECLAIR)	0.0000	0.900	20 ms	0.890	0.0
round-robin	0.0000	0.874	29 ms	0.950	0.0
cost-blind	0.0000	0.943	23 ms	0.959	0.0

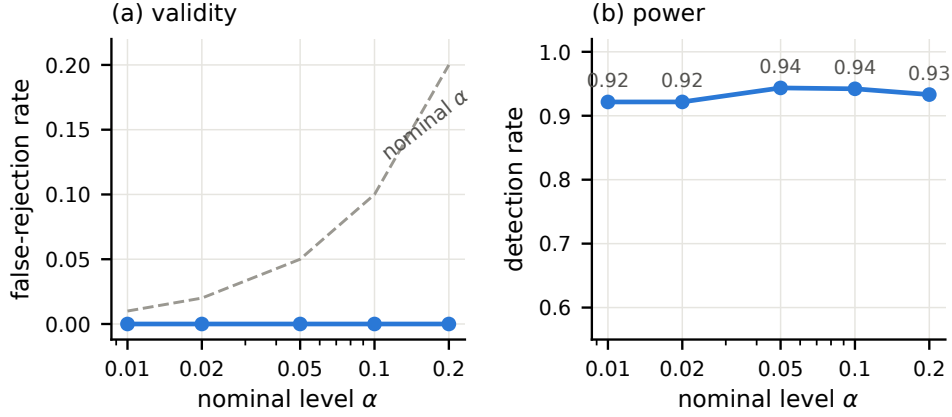


Fig. 3 Validity and power across the operating range (100 replications per level, Kelly routing). (a) Empirical false-rejection of faithful candidates sits at zero under the nominal- α diagonal at every level — the guarantee is an upper bound and holds with the conservative margin the composite null demands. (b) Detection of mutants is flat in α , the empirical face of the logarithmic budget bound of Proposition 4

families than those evaluated. Detection is 87–94% within a sub-second budget. At this testbed’s probe economics the measured tier costs span only a factor 2.4, so cost-normalization separates the routing policies weakly; Section 7.8 isolates the routing question in a controlled cost regime.

7.2 Validity across the operating range

Sweeping $\alpha \in \{0.01, 0.02, 0.05, 0.10, 0.20\}$ (100 replications per level, Kelly routing) yields empirical false-rejection 0.0000 at *every* level ($n \approx 290$ –300 faithful candidates per level), with detection stable at 92–94%; Figure 3 plots both curves. Two remarks.

First, the guarantee of Theorem 2 is an upper bound, not a calibration claim: ECLAIR promises *at most* α , and the observed margin is the structural price of a composite null — tier B cannot false-alarm by proof, and tiers A/C bet against upper-bounded null rates because the true rates are unknowable. Second, the flatness of the

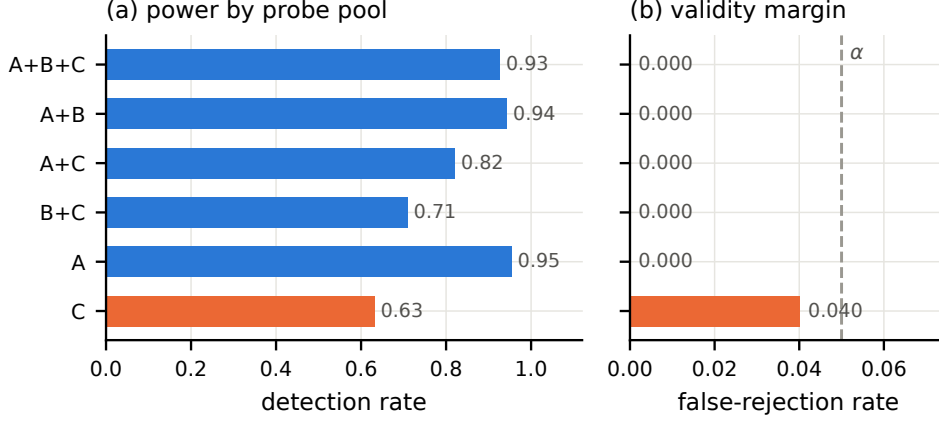


Fig. 4 Probe-tier ablation (100 replications per configuration, Kelly, $\alpha = 0.05$). (a) Detection by probe pool. (b) The validity margin: every pool containing an unentangled tier (blue) records zero false rejections; the pool-entangled cross-model tier alone (orange) stays within α but consumes the entire conservative margin

Table 5 Probe-tier ablation (100 replications/configuration, Kelly, $\alpha = 0.05$).

Tiers	False rej.	Detection	Kill cost
A+B+C	0.0000	0.926	21 ms
A+B	0.0000	0.943	21 ms
A+C	0.0000	0.821	17 ms
B+C	0.0000	0.709	36 ms
A	0.0000	0.954	21 ms
C	0.0401	0.631	71 ms

detection curve is Proposition 4 in action: rejection budgets grow only logarithmically as α tightens.

7.3 Tier ablation: what each probe class buys

Table 5 repeats the testbed with restricted probe pools; Figure 4 visualizes the two headline columns.

Three findings. (i) *Entanglement consumes the conservative margin.* Every configuration containing an unentangled tier records zero false rejections; tier C alone records 0.0401 — still within $\alpha = 0.05$, but with the margin that conservative calibration provides everywhere else fully spent. This is the empirical face of the Remark after Proposition 6: cross-model probes are safe as a supplement, and their calibration must be conditioned on pool composition before they can carry certification alone. (ii) *Hard certificates matter beyond their rate.* Removing tier B (A+C vs. A+B+C) costs

Table 6 External baselines at matched budget (100 replications, $\alpha = 0.05$). Only the e-process framework combines validity with high power; the two fixed-sample-style tests break their nominal level under family transfer and peeking respectively, and the vote-based strategies cannot reject at all.

Method	False rej.	Detection	Pick acc.
ECLAIR (Kelly, all tiers)	0.0000	0.905	0.970
single-shot	0.0000	0.000	0.535
self-consistency majority	0.0000	0.000	0.879
fixed-sample (tier C, $n=8$)	0.0612	0.578	0.714
peeking (tier C, uncorrected)	0.0748	0.601	—
split conformal (tier C)	0.0238	0.480	—

about ten points of detection although tier-B alarms occur on only $\sim 4\%$ of mutant probes. (iii) *A caveat on tier A’s dominance.* Tier A alone detects 95% here, but mutation screening guarantees every mutant is micro-instance-detectable — precisely tier A’s regime. Errors of LLM origin need not be (conventions that diverge only at scale; see below), which is why the tiered design is retained.

7.4 External baselines at matched budget

The routing policies of Table 4 are internal ablations; this section compares against the strategies a practitioner would actually reach for, all given the *same* solver budget per pool and, where applicable, the same calibrated null-rate bound $\bar{p}_0$. *Single-shot* accepts the first candidate unexamined. *Self-consistency majority* solves all candidates on eight shared instances and accepts the one agreeing most often with the per-instance modal value — the standard LLM-ensemble heuristic; it has no rejection concept. *Fixed-sample* runs $n = 8$ cross-model probes per candidate and rejects when the alarm count reaches the exact binomial critical value at level α under $\bar{p}_0$ — a textbook-valid test if its assumptions hold. *Peeking* applies that same binomial test after every probe at nominal α , stopping at the first crossing — the uncorrected sequential usage that testing loops drift into in practice. *Split conformal* rejects when the candidate’s alarm count is extreme against a 200-run faithful calibration sample ($p_{\text{conf}} \leq \alpha$). Table 6 reports 100 replications.

Three comparisons carry the argument. *Against vote-based acceptance:* majority voting picks a faithful model 88% of the time but certifies unfaithful ones the remaining 12% with no warning, rejects nothing, and offers no level to quote; single-shot is the coin flip its 0.535 suggests. *Against fixed-sample testing:* the binomial test is nominally valid under its assumptions, yet realizes $0.0612 > \alpha$ here — the same cross-family transfer and pool entanglement that the e-process absorbs (Section 7.3: even tier-C-only stayed at 0.0401) push a hard accept/reject threshold over its level, because a fixed-sample test has no supermartingale slack to spend. Its peeking variant, the de-facto practice in iterative loops, inflates further to 0.0748 — the canonical optional-stopping failure, here measured rather than merely invoked. *Against conformal:* with an honest faithful-only calibration sample the conformal test keeps its level (0.0238)

Table 7 End-to-end LLM study: intake outcomes per family and model family (3 prompt styles each). Every intake failure is an API hallucination caught by the gate; every intake survivor was proxy-faithful.

Family	routed free	nemotron-ultra	failure modes (intake)
Knapsack+conflicts	2/3	3/3	<code>cp.Binary</code>
Set cover	1/3	3/3	<code>cp.Binary</code> , <code>cp.multiply</code>
Graph colouring	2/3	3/3	<code>IntVar(domain=...)</code>
Gen. assignment	0/3	3/3	<code>cp.Var (×2)</code> , <code>cp.binary_var</code>
On-time selection	1/3	2/3	<code>import cp, NewBoolVar</code> , runtime <code>None</code>
Total	6/15	14/15	

but detects barely half of what ECLAIR does at the same budget (0.480 vs. 0.905), because a fixed probe count cannot concentrate effort where evidence is accumulating. Matched budget, same probes, same calibration information — the difference is purely the accounting.

7.5 End-to-end study with real LLMs

Thirty candidates were generated from natural-language specifications of the five families: two model families accessed through OpenRouter (a free-tier routed model and `nvidia/nemotron-3-ultra-550b`) $\times$ three prompt styles (direct; restate-then-model; expert persona) $\times$ per-style temperatures. Generated code is executed in a sandboxed namespace; an intake gate requires each candidate to build and solve two micro-instances. For scoring only, a proxy faithfulness label compares each candidate against the enumeration oracle on twelve micro-instances. All prompts, raw responses, and the response cache are included in the supplementary material, and this paragraph constitutes the documentation of LLM use in this study’s methods.

Generation. 20/30 candidates passed intake (Table 7). All ten failures were *loud*: API hallucinations importing conventions from other modelling ecosystems — `cp.Binary` and `cp.binary_var` (PuLP/DOcplex idioms), `cp.Var`, `cp.multiply`, `IntVar(domain=...)` (Choco-style), `Model().NewBoolVar` (raw CP-SAT API), a bare `import cp` — each caught by the gate at negligible cost. The failure mass is unevenly distributed: the large dedicated model produced valid CPMpy for 14/15 prompts, while the free-tier routed model managed 6/15, failing on entire families (generalized assignment 0/3) — candidate diversity across model families is what keeps such correlated failure from starving a pool.

Certification. All 20 intake survivors were proxy-faithful, so the certified pools are all-null: the correct behaviour is to reject nothing, and that is what happened — 0/20 false rejections under every routing policy, the certified pick was faithful in all five families, no abstentions, and no e-BH false discoveries. The guarantee of Theorem 2 thus survives contact with real generated models, complementing the mutation testbed in which the same machinery demonstrably detects 87–94% of actual faults.

Two honest observations shape the outlook. On well-pinned specifications, these two model families fail loudly (crashes) rather than silently (wrong models) — so the certification layer’s detection power is insurance whose premium, by Proposition 4,

Table 8 Robustness sweeps (100 replications per row, Kelly, $\alpha = 0.05$).

Sweep	Configuration	False rej.	Detection
family	set cover	0.0000	0.990
family	generalized assignment	0.0000	0.743
family	on-time selection	0.0000	0.971
transfer	reversed calibration split	0.0000	0.804
pool	$k = 4$	0.0000	0.934
pool	$k = 10$	0.0000	0.893
prior	$q_{\text{prior}} = 0.3$	0.0000	0.741
prior	$q_{\text{prior}} = 0.7$	0.0000	0.920

is logarithmically cheap. And provoking the silent-error regime in LLM candidates requires deliberately ambiguous, convention-laden specifications (the colouring objective convention — “number of colours” vs. “maximum colour index” — is the template), which is exactly where ECLAIR’s abstention-with-evidence output becomes the interesting object of study; we return to this in Section 10.

7.6 Robustness: families, transfer direction, pool composition

Three sweeps probe whether the headline numbers are artifacts of a particular experimental slice (100 replications each, Kelly, $\alpha = 0.05$; Table 8). *Per family*: validity is exact in every evaluation family separately; detection varies with family difficulty (0.99 for set cover and 0.97 for on-time selection against 0.74 for generalized assignment, whose cost-matrix mutations produce the subtlest optimum shifts) — power is problem-dependent, the guarantee is not. *Reversed calibration split*: recalibrating on the three former evaluation families and evaluating on knapsack+colouring yields false rejection 0.0000 with detection 0.804 ($n = 309$ faithful) — cross-family transfer of the conservative bounds is bidirectional, not an artifact of which families happened to calibrate. *Pool size and routing prior*: across $k \in \{4, 6, 10\}$ and routing priors $q_{\text{prior}} \in \{0.3, 0.5, 0.7\}$, false rejection stays at 0.0000 while detection moves only through the router’s aggressiveness (a pessimistic prior under-probes, 0.74; a suspicious one probes harder, 0.92) — the router’s belief state touches power alone, never the level, exactly as Theorem 2 promises for any predictable routing.

7.7 Probing the silent-error regime: ambiguous specifications

The main LLM study’s lesson — loud failures, faithful survivors — raises the question of whether *silent* misreadings can be elicited at all. We rephrased three specifications to plant boundary ambiguities, fixed one canonical reading in the reference checker, and regenerated candidate pools (both model families, three prompt styles each): colouring with the objective convention sentence *removed* (“minimize the number of colors used”), knapsack with “the selected items together must be lighter than the capacity” (strict vs. non-strict), and on-time selection with “every selected job is finished before its deadline” (strict vs. non-strict completion).

The outcome is a negative result we report with its explanation: 10/18 generations passed intake, and *all ten* implemented the canonical reading — no silent other-reading model appeared. Certification behaved correctly throughout (0/10 false rejections, canonical pick in 3/3 families, no abstention). Two mechanisms account for the miss. First, one planted ambiguity is value-equivalent at the optimum: a model minimizing the *count* of distinct colours and one minimizing $1 + \max$ index return the same optimal value on every instance, by precisely the renumbering argument of Appendix A — the readings differ only off-optimum, where no value-level probe looks. Second, for the strict / non-strict boundary pairs, current models normalize aggressively to the conventional reading ($\leq$): the training distribution of textbook formulations evidently dominates the literal text. The constructive lesson for the elicitation agenda (Section 10) is that silent LLM errors are unlikely to hide in boundary conventions; structural underspecification — omitted side constraints, ambiguous aggregation, units — is the more promising hunting ground, and value-equivalent ambiguities require solution-level (not optimum-level) probes, which the framework accommodates as additional tier-A checkers.

7.8 Routing under heterogeneous probe costs

The CP-SAT testbed’s probes all cost single-digit milliseconds, which compresses the routing question. To isolate it, a controlled simulation reproduces the certification layer with the measured alarm-rate structure but a 20:1 probe-cost spread (expensive near-exact oracles vs. cheap noisy checks — the regime of industrial instances, of costly solver calls, and of LLM-as-judge probes priced per token). Over 500 replications (Figure 5): Kelly routing detects 86% of unfaithful candidates at mean cost 12.2 budget units per kill, against 60% at 16.6 for round-robin and 77% at 22.6 for cost-blind selection — while false rejection stays at 0.000–0.003, below α for every policy. Routing gains scale with probe-cost heterogeneity; validity is routing-invariant, as Theorem 2 requires.

8 Deployment guidance

Because every quantity in the framework is operational, the knobs a deploying team must set reduce to four, each with a concrete rule.

The level α .

By Proposition 4 and Figure 3, tightening α costs only logarithmic budget and essentially no detection — so α should be chosen by the downstream stakes (what fraction of wrongly discarded faithful models, and wrongly certified survivors under e-BH, the application tolerates), not by compute anxiety. We used 0.05 throughout; 0.01 behaved identically.

The calibration corpus.

Validity consumes exactly one number per noisy tier: an upper confidence bound $\bar{p}_0$ on the null alarm rate. With zero observed false alarms in n calibration trials, the

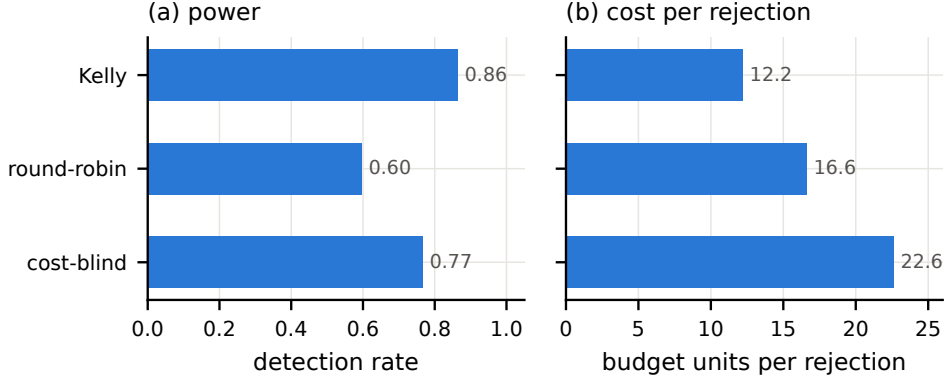


Fig. 5 Routing under a 20:1 probe-cost spread (500 replications, $\alpha = 0.05$; measured alarm-rate structure, controlled costs). Kelly routing dominates on both detection (a) and budget per rejection (b); false-rejection stays below α under every policy, as validity is routing-invariant

Clopper–Pearson bound scales as $\bar{p}_0 \approx 3.7/n$ ($n=100$: 0.036; $n=240$: 0.015; $n=1000$: 0.0037), and a smaller $\bar{p}_0$ buys a larger alarm multiplier $p_1/\bar{p}_0$ — calibration effort converts directly into per-alarm evidence. Known-faithful models for the corpus are the cheap part: hand-written reference models on any families adjacent to the deployment domain suffice, and our results show the bounds transfer across families when used conservatively.

The budget.

The testbed’s operating point — ~ 80 mean-cost probes per pool of six — put the marginal probe well past the detection plateau. The budget-to-rejection bound gives the back-of-envelope sizing rule: expect $\sim \log(1/\alpha)/\Delta$ probes per unfaithful candidate, with Δ the per-probe expected log-evidence (≈ 1.5 for our calibrated tier A under $\hat{p}_1 \approx 0.6$), so single-digit probes per rejection — consistent with the measured 20 ms kill costs against 5 ms probes.

What to log.

The evidence ledger — every probe, its tier, instance seed, outcome, and e-factor — is the audit object: it makes each rejection reproducible (replay the counterexample), each certification quantifiable (survived evidence at level α), and each abstention actionable (which readings died on which probes). Teams wrapping an existing agentic modelling loop [3, 13] need to intercept only two things: candidate model source and solver calls.

9 Threats to validity

Internal. The statistical guarantee rests on Assumption 1; the ablation localizes exactly where its tier-A/C half is fragile (pool-entangled probes carrying sole weight) and the

baselines show the failure is not hypothetical — the same miscalibration that leaves the e-process at 0.040 pushes a fixed-sample threshold to 0.061. Tier-B validity is by proof, but only for relations proved against the *specification*; a relation read off a formulation can be invalid (Section 6).

Construct. Mutation screening makes every testbed fault micro-detectable, biasing tier importance toward A (Section 7.3); proxy faithfulness labels in the LLM studies are themselves enumeration-based and share this bias. The ambiguity study is the counterweight, since other-reading models differ precisely where enumeration on canonical micro-instances says they do.

External. Two model families, five problem families, one modelling language: claims about “LLM modellers” beyond these are extrapolation. The error taxonomy (Table 7) will age as models improve; the framework’s premise — that whatever errors remain be caught with a guarantee — does not depend on the taxonomy’s contents.

10 Limitations and outlook

Calibration scope. Validity rests on Assumption 1; our evidence that conservative calibration transfers across problem families is empirical (zero false rejections under family transfer), not a theorem, and tier-C calibration should be conditioned on pool composition before cross-model evidence carries substantial weight — the ablation shows the margin, though never the bound, giving way. *Screened mutants.* The testbed’s faults are guaranteed micro-detectable by construction; LLM-native silent errors need not be, and eliciting them requires deliberately ambiguous specifications — the planned abstention study, which also connects certification to conversational elicitation. *Checker provenance.* Our enumeration checkers are hand-written and independent; a fully LLM-generated pipeline must calibrate checker error as tier-A noise, which the framework accommodates but this study does not exercise. *Breadth.* Five families suffice to exercise every mechanism; scaling the probe library toward CSPLib/NL4Opt breadth, and stating the MR validity results for global constraints (`alldifferent`, `cumulative`) in the generality they deserve, are the natural next steps. *Beyond one modelling language.* Nothing in the framework is CPMpy-specific: candidates are opaque functions from instances to optima, so MiniZinc, MILP, or SMT candidates certify identically — indeed a heterogeneous pool (the same specification modelled in CP and MILP) strengthens tier C precisely because formalism diversity decorrelates errors. What each new language requires is an exact solve at micro/mid scale and, for tier B, relations proved against the specification; the three relation classes of Section 6 are formalism-independent. *Eliciting silent errors.* The ambiguity study (Section 7.7) shows boundary conventions are normalized away by current models and value-equivalent readings are invisible to optimum-level probes; the elicitation agenda should target structural underspecification, and the probe pool should grow solution-level checkers where readings coincide in value but not in argmax.

11 Conclusion

ECLAIR turns the verification of LLM-generated constraint models from a heuristic detection exercise into sequential hypothesis testing with explicit guarantees: anytime-valid false-rejection control under optional stopping and adaptive, budget-aware probe routing; FDR control across a generate–test–repair tournament; and an auditable evidence ledger for every decision, including abstention. On real CP-SAT probes the guarantee held exactly at every operating level while nine of ten injected faults were caught within sub-second budgets, and the framework certified real LLM-generated models without error. The broader claim is methodological: solver-in-the-loop verification of generative models is, at bottom, a sequential inference problem, and the e-process toolkit fits it exactly.

Statements and Declarations

Funding. The authors did not receive support from any organization for the submitted work.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Data availability. All code, calibration data, experimental results, and cached LLM generations are available in the supplementary material.

Use of large language models. LLMs are the object of study; their use in the experiments (models, prompting, decoding parameters, caching) is documented in Section 7.5.

Appendix A Validity proofs for the probe library

Throughout, $\mathcal{X}(I)$ denotes the specification’s feasible set on instance I , f_I its objective, and $v^*(I) = \text{opt}_{x \in \mathcal{X}(I)} f_I(x)$. Because tier-B probes solve the *candidate* exactly on both sides, and a faithful candidate satisfies $v_m \equiv v^*$, it suffices to prove each relation for v^* . Three lemmas cover all but one relation.

Lemma 7 (Relaxation) *If $\mathcal{X}(I) \subseteq \mathcal{X}(I')$ and $f_{I'} = f_I$, then $v^*(I') \geq v^*(I)$ for maximization and $v^*(I') \leq v^*(I)$ for minimization.*

Proof The optimum over a superset is no worse: any x optimal for I is feasible for I' with the same objective value. $\square$

Lemma 8 (Objective dominance) *If $\mathcal{X}(I') = \mathcal{X}(I)$ and $f_{I'}(x) \geq f_I(x)$ for all $x \in \mathcal{X}(I)$, then $v^*(I') \geq v^*(I)$, for maximization and minimization alike.*

Proof For maximization, dominate the maximizer of I ; for minimization, $\min_x f_{I'}(x) \geq \min_x f_I(x)$ since every point’s value rose. $\square$

Lemma 9 (Symmetry) *If π is a bijection of the decision index set and I' is I with all indexed data consistently relabeled by π , then $x \mapsto x \circ \pi^{-1}$ is a bijection $\mathcal{X}(I) \rightarrow \mathcal{X}(I')$ preserving objective values; hence $v^*(I') = v^*(I)$.*

Proof Every constraint and objective term of the specification refers to indices only through the relabeled data, so feasibility and value are preserved in both directions. $\square$

Knapsack with conflicts (max).

Capacity increase relaxes the single budget constraint (Lemma 7: $\uparrow$). Deleting a conflict pair removes a constraint (Lemma 7: $\uparrow$). Item permutation, with the conflict list relabeled by the same permutation, is Lemma 9.

Set cover (min).

Deleting an element removes one covering constraint (Lemma 7: $\downarrow$). Raising one set's cost is Lemma 8 on a minimization ($\uparrow$). Permuting the set collection (costs and coverage lists together) is Lemma 9.

Graph colouring (min).

The specification minimizes the number of colours; the implemented objective, $1 + \max_i c_i$ over proper colourings with colours in $\{0, \dots, n-1\}$, agrees with it at the optimum because any proper colouring with k distinct colours can be renumbered onto $\{0, \dots, k-1\}$ without violating properness. Edge deletion removes a disequality constraint (Lemma 7: $\downarrow$); edge addition adds one, shrinking the feasible set (Lemma 7 applied with the roles of I, I' exchanged: $\uparrow$); vertex relabeling is Lemma 9.

Generalized assignment (min).

Raising one machine's capacity relaxes its load constraint (Lemma 7: $\downarrow$). Raising one cost entry is Lemma 8 ($\uparrow$). Task permutation (sizes and cost rows together) is Lemma 9.

On-time job selection (max).

This is the delicate family. The specification selects $S \subseteq \{1, \dots, n\}$ maximizing $\sum_{j \in S} w_j$ subject to *schedulability*: some single-machine order completes every job of S by its deadline. The classical characterization states that S is schedulable iff processing S in nondecreasing-deadline (EDD) order is feasible, iff

$$\sum_{i \in S: d_i \leq d_j} p_i \leq d_j \quad \text{for every } j \in S. \quad (\text{A1})$$

Weight increase is Lemma 8 ($\uparrow$). *Processing-time decrease*: in any fixed order every completion time weakly decreases, so every schedulable S remains schedulable; the feasible set (in the schedulability sense) grows, and Lemma 7 gives $\uparrow$. *Deadline increase* $d_k \mapsto d_k + \delta$: note that the index sets in (A1) change with the deadline order, so monotonicity of the inequality system is not syntactic. Argue at the schedulability level instead: fix a schedule witnessing feasibility of S under the old deadlines; completion times are unchanged and deadlines are pointwise no smaller, so the same schedule

witnesses feasibility under the new deadlines. Hence every schedulable set remains schedulable, and Lemma 7 gives $\uparrow$; the candidate, if faithful, satisfies (A1) on both sides because (A1) is equivalent to schedulability instance by instance. Job permutation (processing times, deadlines, and weights together) is Lemma 9. $\square$

Appendix B Natural-language specifications and prompts

For reproducibility, the exact specification texts and the prompt template of the end-to-end study (Section 7.5). Candidates were requested as a single Python function `build_model(params)` returning a CPMpy model with the objective set, under three style hints — *direct* (“Write the model directly.”, temperature 0.3), *stepwise* (“First restate the decision variables, constraints and objective in one short paragraph each, then write the code.”, 0.7), and *expert* (“You are an expert constraint-programming modeller known for scrupulously faithful formalizations. Write the model.”, 0.5).

Template.

```
Translate the following optimization problem into a CPMpy model.
{SPECIFICATION}
{STYLE HINT}
Requirements: provide a single Python function build_model(params) that
returns a cpmPy.Model with the objective set (use model.maximize(...) or
model.minimize(...)); use only the cpmPy library (imported as cp) and the
Python standard library; the function must work for ANY params dict matching
the schema; reply with ONLY a Python code block.
```

Specification texts (abridged schema lines).

Knapsack with conflicts: “You must select a subset of n items. Item i has weight $w[i]$ and value $v[i]$. The total weight of selected items must not exceed the capacity cap . Some pairs of items conflict: for each pair (i, j) in the list `conflicts`, items i and j must not both be selected. Maximize the total value of the selected items.” Set cover: “There are $nelem$ elements, numbered $0..nelem - 1$, and a collection of sets. Set s covers the elements listed in `sets[s]` and costs `cost[s]`. Choose a sub-collection of sets so that every element is covered by at least one chosen set. Minimize the total cost of the chosen sets.” Graph colouring: “Color the n vertices of an undirected graph, using color indices from 0 to $n - 1$, so that the two endpoints of every edge receive different colors. Minimize the number of colors used. IMPORTANT convention: the objective value must equal 1 plus the maximum color index used.” Generalized assignment: “Assign each of T tasks to exactly one of M machines. Task t has size $p[t]$; machine m has capacity $cap[m]$: the total size of tasks assigned to machine m must not exceed $cap[m]$. Assigning task t to machine m costs $cost[t][m]$. Minimize the total assignment cost.” On-time job selection: “A single machine processes jobs one at a time. Job j has processing time $p[j]$, deadline $d[j]$ and weight $wt[j]$. Select a subset of jobs that can ALL be completed by their deadlines when the selected jobs are processed in non-decreasing deadline order (equivalently: a selected set is feasible if and only if, for

every selected job j , the sum of processing times of selected jobs i with $d[i] \leq d[j]$ is at most $d[j]$). Maximize the total weight of the selected jobs.”

The colouring convention sentence is deliberate: without it, “minimize the number of colors” admits the objective $|\{c_i\}|$ as a faithful reading, and the two readings differ only off-optimum — the template for the planned ambiguous-specification study (Section 10).

Appendix C The mutation operator catalogue

Each operator is a deterministic function of instance parameters, drawn once per candidate and applied identically to every instance the candidate ever sees (a wrong *model*, not a wrong instance). Descriptors are sampled uniformly over the applicable (operator, target) pairs of Table 2; positional indices are drawn once and taken modulo the current instance’s dimension, which is what makes them instance-generic.

scalar shift a named scalar bound $b \mapsto \max(1, b + \delta)$, $\delta \in \{\pm 1, \pm 2, \pm 3\}$ (misread constant).

vector entry one positionally selected entry of a named numeric vector shifted by δ as above (misread datum).

vector rotation a named vector rotated by one position — the data-wiring misalignment in which item i receives item $(i+1)$ ’s datum; the mutation class that only the permutation MR and value-level oracles can see.

row drop one positionally selected structural row (conflict pair, edge, coverage list) deleted (missed constraint).

family half-drop the second half of a structural family deleted (systematically missed constraint class).

matrix entry one positionally selected entry of a cost matrix shifted by δ .

operator flip one constraint family’s comparison direction inverted at the builder level ($\leq \leftrightarrow \geq$; for colouring, disequality to equality) — the class that breaks monotone MRs and is otherwise invisible to agreement checks when the flipped model is infeasible-prone.

Screening. A drawn mutant is kept only if its exact optimum differs from the specification’s brute-forced optimum on at least one of six random micro-instances (up to 40 redraws); this discards semantically equivalent mutants, the standard practice in mutation testing, and guarantees ground-truth labels for every experiment in Section 7.

Appendix D Extended numerical tables

For re-derivation without running code, the full operating-range table behind Figure 3 (100 replications per level, Kelly routing; kill cost is mean measured solver time to reject a mutant):

Calibrated betting multipliers implied by Table 3 (equation (1)): tier A alarm $\times 30.9$, pass $\times 0.535$; tier C alarm $\times 3.00$, pass $\times 0.607$; tier B pass $\times 1$, alarm $= \infty$. Rejection threshold at $\alpha = 0.05$: wealth 20, i.e. log-threshold 3.00.

Table D1 Full α -sweep.

α	false rej.	n faithful	detection	n mutants	kill cost
0.01	0.0000	300	0.923	300	30.5 ms
0.02	0.0000	285	0.937	315	30.0 ms
0.05	0.0000	297	0.924	303	22.8 ms
0.10	0.0000	301	0.930	299	19.4 ms
0.20	0.0000	293	0.964	307	20.7 ms

The kill-cost column is Proposition 4 measured: tightening the guarantee from $\alpha = 0.20$ to $\alpha = 0.01$ — a $20\times$ stronger error bound — raises the mean solver cost of a rejection from ~ 21 ms to ~ 30 ms, a factor of ~ 1.5 , in line with the predicted $\log(1/0.01)/\log(1/0.20) \approx 2.9$ ratio of log-thresholds attenuated by hard-certificate rejections whose cost is α -independent.

Appendix E Complete generation record

Every LLM generation across both studies (main: Section 7.5; ambig: Section 7.7), in request order. “free” is the free-tier routed model; “nemotron” is `nvidia/nemotron-3-ultra-550b`; error classes abbreviate the exception type raised at the intake gate (`AttrErr: AttributeError`, `ModErr: ModuleNotFoundError`, `TypeErr: TypeError`, `SynErr: SyntaxError`); the raw responses ship in the supplementary cache.

References

- [1] Michailidis, K., Tsouros, D., Guns, T.: Constraint modelling with LLMs using in-context learning. In: 30th International Conference on Principles and Practice of Constraint Programming (CP 2024). LIPIcs, vol. 307 (2024). <https://doi.org/10.4230/LIPIcs.CP.2024.20>
- [2] Shi, W., Liu, M., Zhang, W., Shi, L., Jia, F., Ma, F., Zhang, J.: ConstraintLLM: A neuro-symbolic framework for industrial-level constraint programming. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP) (2025). <https://arxiv.org/abs/2510.05774>
- [3] Szeider, S.: CP-Agent: Agentic constraint programming (2025). <https://arxiv.org/abs/2508.07468>
- [4] Ramdas, A., Grünwald, P., Vovk, V., Shafer, G.: Game-theoretic statistics and safe anytime-valid inference. *Statistical Science* **38**(4), 576–601 (2023) <https://doi.org/10.1214/23-STS894>
- [5] Shafer, G.: Testing by betting: A strategy for statistical and scientific communication. *Journal of the Royal Statistical Society: Series A* **184**(2), 407–431 (2021) <https://doi.org/10.1111/rssa.12647>

Table E2 All 48 generations.

Study	Family	Model	Style	Intake	Outcome / error
main	knapsack_conflicts	free	direct	fail	AttrErr
main	knapsack_conflicts	free	stepwise	ok	faithful
main	knapsack_conflicts	free	expert	ok	faithful
main	knapsack_conflicts	nemotron	direct	ok	faithful
main	knapsack_conflicts	nemotron	stepwise	ok	faithful
main	knapsack_conflicts	nemotron	expert	ok	faithful
main	set_cover	free	direct	fail	AttrErr
main	set_cover	free	stepwise	ok	faithful
main	set_cover	free	expert	fail	AttrErr
main	set_cover	nemotron	direct	ok	faithful
main	set_cover	nemotron	stepwise	ok	faithful
main	set_cover	nemotron	expert	ok	faithful
main	coloring	free	direct	ok	faithful
main	coloring	free	stepwise	ok	faithful
main	coloring	free	expert	fail	TypeErr
main	coloring	nemotron	direct	ok	faithful
main	coloring	nemotron	stepwise	ok	faithful
main	coloring	nemotron	expert	ok	faithful
main	gen_assignment	free	direct	fail	AttrErr
main	gen_assignment	free	stepwise	fail	AttrErr
main	gen_assignment	free	expert	fail	AttrErr
main	gen_assignment	nemotron	direct	ok	faithful
main	gen_assignment	nemotron	stepwise	ok	faithful
main	gen_assignment	nemotron	expert	ok	faithful
main	ontime_selection	free	direct	fail	ModErr
main	ontime_selection	free	stepwise	fail	AttrErr
main	ontime_selection	free	expert	ok	faithful
main	ontime_selection	nemotron	direct	ok	faithful
main	ontime_selection	nemotron	stepwise	ok	faithful
main	ontime_selection	nemotron	expert	fail	AttrErr
ambig	coloring	free	direct	fail	SynErr
ambig	coloring	free	stepwise	fail	ModErr
ambig	coloring	free	expert	fail	AttrErr
ambig	coloring	nemotron	direct	ok	canonical
ambig	coloring	nemotron	stepwise	fail	AttrErr
ambig	coloring	nemotron	expert	ok	canonical
ambig	knapsack_conflicts	free	direct	fail	no code block
ambig	knapsack_conflicts	free	stepwise	fail	AttrErr
ambig	knapsack_conflicts	free	expert	fail	no code block
ambig	knapsack_conflicts	nemotron	direct	ok	canonical
ambig	knapsack_conflicts	nemotron	stepwise	ok	canonical
ambig	knapsack_conflicts	nemotron	expert	ok	canonical
ambig	ontime_selection	free	direct	ok	canonical
ambig	ontime_selection	free	stepwise	ok	canonical
ambig	ontime_selection	free	expert	fail	AttrErr
ambig	ontime_selection	nemotron	direct	ok	canonical
ambig	ontime_selection	nemotron	stepwise	ok	canonical
ambig	ontime_selection	nemotron	expert	ok	canonical

- [6] Grünwald, P., Heide, R., Koolen, W.M.: Safe testing. *Journal of the Royal Statistical Society Series B: Statistical Methodology* **86**(5), 1091–1128 (2024) <https://doi.org/10.1093/jrsssb/qkae011>
- [7] Ville, J.: *Étude Critique de la Notion de Collectif*. Gauthier-Villars, Paris (1939)
- [8] Kelly, J.L.: A new interpretation of information rate. *Bell System Technical Journal* **35**(4), 917–926 (1956) <https://doi.org/10.1002/j.1538-7305.1956.tb03809.x>

- [9] Wang, R., Ramdas, A.: False discovery rate control with e-values. *Journal of the Royal Statistical Society Series B: Statistical Methodology* **84**(3), 822–852 (2022) <https://doi.org/10.1111/rssb.12489>
- [10] Guns, T.: Increasing modeling language convenience with a universal n-dimensional array: CPy as python-embedded example. In: *Proceedings of the 18th Workshop on Constraint Modelling and Reformulation (ModRef)* (2019)
- [11] Perron, L., Furnon, V.: OR-Tools CP-SAT solver. Google, <https://developers.google.com/optimization> (2024)
- [12] Michailidis, K., Tsouros, D., Guns, T.: DCP-Bench-Open: Evaluating LLMs for constraint modelling of discrete combinatorial problems. CP-Bench version accepted at ECAI 2025 (2025). <https://arxiv.org/abs/2506.06052>
- [13] Szeider, S.: MCP-Solver: Integrating language models with constraint programming systems (2025). <https://arxiv.org/abs/2501.00539>
- [14] Chen, T.Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T.H., Zhou, Z.Q.: Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys* **51**(1), 4–1427 (2018) <https://doi.org/10.1145/3143561>
- [15] DeMillo, R.A., Lipton, R.J., Sayward, F.G.: Hints on test data selection: Help for the practicing programmer. *Computer* **11**(4), 34–41 (1978) <https://doi.org/10.1109/C-M.1978.218136>
- [16] Jia, Y., Harman, M.: An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* **37**(5), 649–678 (2011) <https://doi.org/10.1109/TSE.2010.62>
- [17] Vovk, V., Wang, R.: E-values: Calibration, combination and applications. *The Annals of Statistics* **49**(3) (2021) <https://doi.org/10.1214/20-AOS2020>
- [18] Waudby-Smith, I., Ramdas, A.: Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology* **86**(1), 1–27 (2024) <https://doi.org/10.1093/jrssb/qkad009>
- [19] Clopper, C.J., Pearson, E.S.: The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika* **26**(4), 404–413 (1934) <https://doi.org/10.1093/biomet/26.4.404>
- [20] Freund, Y., Schapire, R.E.: A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences* **55**(1), 119–139 (1997) <https://doi.org/10.1006/jcss.1997.1504>
- [21] Cesa-Bianchi, N., Lugosi, G.: *Prediction, Learning, and Games*. Cambridge University Press, ??? (2006). <https://doi.org/10.1017/CBO9780511546921>